Technical Walkthrough: FCY Payment Processor

Go | PostgresSQL | Docker

March 19, 2026

Paul O. Afolabi
Software Engineer

Problem Framing

- Build a multi-currency processor for internal and external transfers.
- The processor should guarantee correctness of balances, postings, and auditable transaction trail.
- Prioritize pragmatic production judgment under time constraints.
- Tech stack: Go, PostgreSQL, Docker Compose.

Solution Architecture

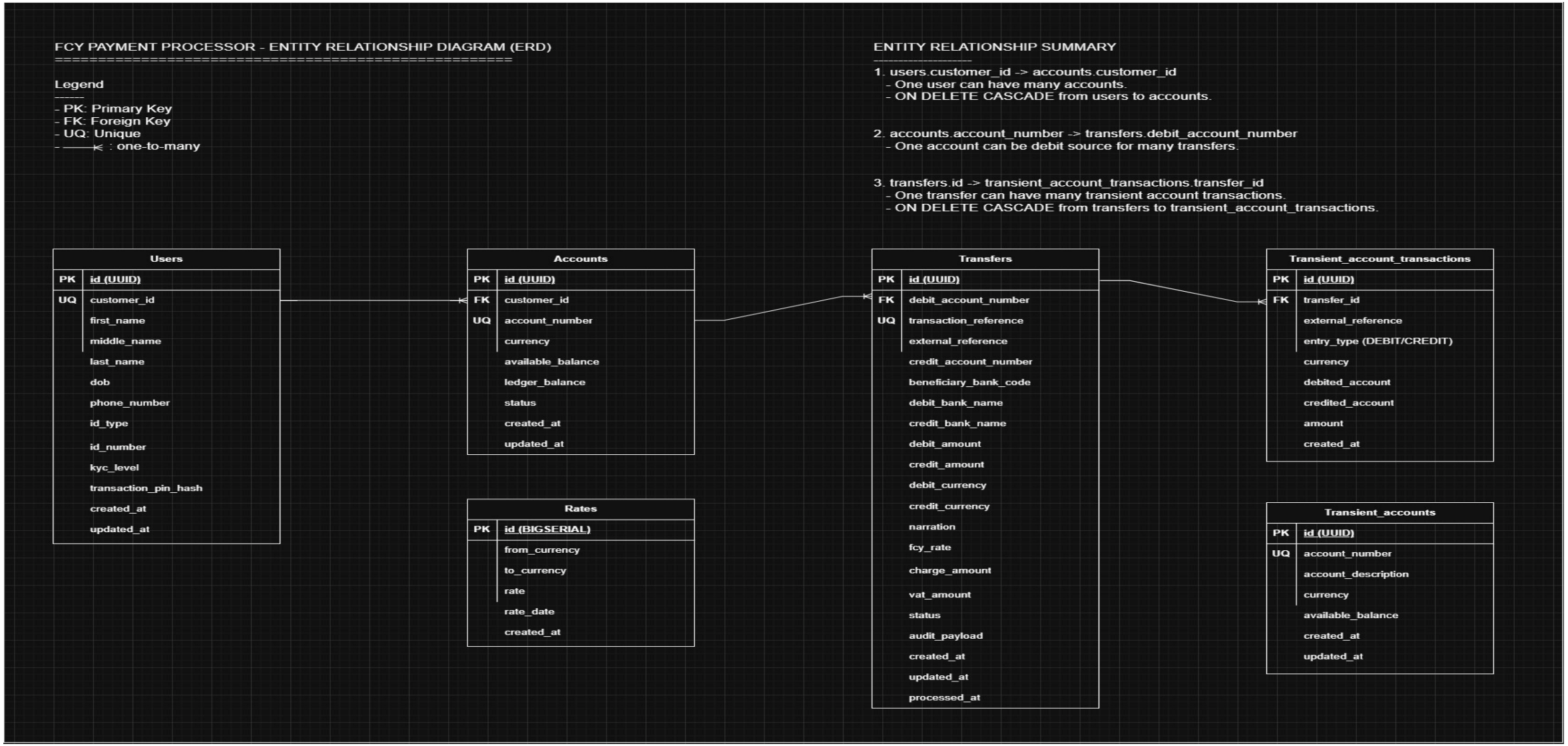
Clean Architecture + Repository Pattern was adopted as a suitable design pattern for the solution.

- HTTP Adapters → Use Cases (IService) → IRepository → Repository Implementations → PostgreSQL

- Dependency inversion implementation to separate high-level logic from low level implementation.

This structure improves testability, maintainability, and safe evolution of the payment system.

Solution Architecture – ERD, Domain and Data Model



Solution Architecture – Sequence Diagram

PAYMENT PROCESSOR SEQUENCE FLOW AND DIAGRAM

==========

Note: API authentication and authorization is conducted for every interaction between the Channel and the API layers, hence the bi-directional communication.

1. User selects account number to be debited and account enquiry is done. User account details are retuned.

2. User provides beneficiary account details and account enquiry is done. If beneficiary bank code is not intra, request is routed to 3rd party infra. Beneficiary account details are returned.

3. User enters amount and triggers get rate operation and get charges operation. Exchange rate, charges, VAT, amount beneficiary receives is returned.

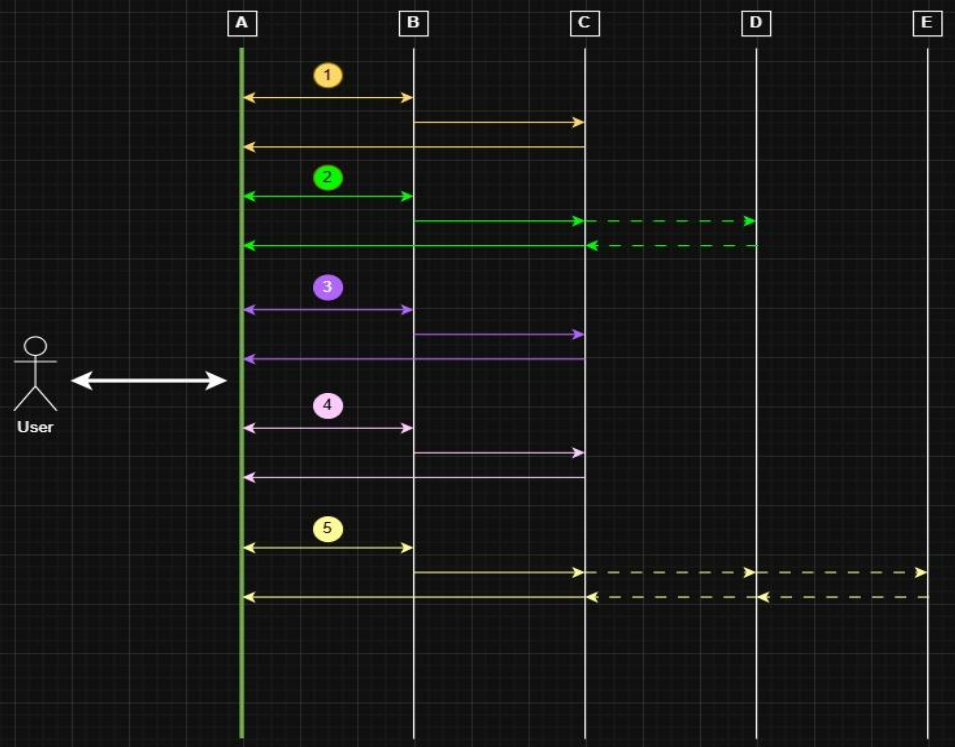
4. User proceeds to provide narration and other analytical details of the transaction and then provides PIN. This triggers verifyPin operation. Returns PIN validity response. Here, hashing and salting is leveraged for hightened security.

5. User hits the send button and triggers the /internal or /external transfer operation depending on the bankcode provided. Payment request is picked up from the Queue, persisted into the Db with PENDING status and processed. Returns transaction reference, payment reference, etc, upon successful payment. Else, it returns a specific response message, reverses transaction appropriately into the User's account using transaction reference and updates transaction status to FAILED. Payment ends. For successful payments, internal settlement job (Queue service) is triggered to debit suspense account and credit charges, VAT internal GL accounts, and external GL account(s) if they exist. Transaction status is then updated to CLOSED.

==========

INFRASTRUCTURAL LAYERS

- A Channel Layer
- B Payment Processor API Layer
- C PostgresSQL Db Layer (Data management and Persistence)
- D 3rd Party Application Layer
- E Beneficiary Bank



API Endpoints

1. POST /create-user – Creates a customer record. Generates and returns customerId
2. POST /create-account – Creates FCY account using the customerId generated
3. GET /get-account – Fetches provided account details if it exists
4. POST /deposit-funds – Deposits funds for internal, sender account
5. GET /get-participant-banks – Fetches external banks' details supported by the payment processor
6. GET /get-rates – Fetches all maintained currency pairs and rates from the database
7. POST /get-rate – Fetches specific maintained currency pair and exchange rate
8. GET /get-charges – Computes and returns detailed charges per amount and currency provided
9. POST /convert-fcy-amount – Computes amount in sender's currency relative to beneficiary currency
10. POST /verify-pin – Verifies user's transaction PIN
11. POST /transfer-funds - Debits senders accounts and credits beneficiary's account.

    (beneficiaryBankCode=100100 for internal transfers)

Transfer flow - Internal

1. **Validation**: Validate request, account states/currencies, and transaction PIN
2. **Computation and Persistence**: Compute FX + charges/VAT and persist transfer as PENDING
3. **Posting**: debit sender (full amt) => credit GL account (full amt) => debit GL account (txn amt) => credit beneficiary account (txn amt)
4. **Update and Settlement**: Status transitions to SUCCESS, then CLOSED after transaction settlement into charges and VAT GL accounts
5. **Response**: Transaction details are returned to the user/channel

Transfer flow - External

1. **Routing**: Route by beneficiary bank code (non-Grey – 100100 - means external transfer)
2. **Validation**: Validate request, account status/currency, transaction PIN, participant bank support and resolve destination external GL by currency
3. **Computation and Persistence**: Compute FX + charges/VAT and persist transfer as PENDING
4. **Posting**: debit sender (full amt) => credits internal GL account (full amt) => debit internal GL account (txn amt) => credit external GL account of the same currency as beneficiary currency (txn amt)
5. **Update and Settlement**: Persist external reference for traceability. Status transitions to SUCCESS, then CLOSED after transaction settlement into charges and VAT GL accounts
6. **Response**: Transaction details are returned to the user/channel

Amount Handling, Consistency, Atomicity and Failure Handling

1. All money math uses decimal.Decimal to accommodate large amount values.
2. No approximations of figures. I explicitly round initial deposits to 2 decimal places at service level, and monetary persistence (Dr/Cr) uses 2dp DB numeric columns.
3. Charge computed in USD with min/max bounds; converted to debit currency when needed.
4. Transaction posting is atomic using DB transaction to improve financial reliability.
5. Settlement is decoupled as second leg for resilience and recoverability.

Security, Concurrency, and Operational Readiness

1. Channel-level Basic Auth middleware with constant-time credential compare.
2. Transaction PIN hashed with bcrypt and verified before transfer.
3. Structured logging and startup/transfer timing instrumentation.
4. DB connection pool tuned for concurrent workloads.
5. Leveraged goroutines for parallel account lookups, charge/rate computations, and startup initialization to reduce latency and improve throughput.

Testing Strategy

- Service-focused unit tests with validation and happy-path coverage.
- Core test packages currently pass (14 tests total).
- Strong focus on business-rule correctness over broad integration depth.

Trade-offs and Risks

- Current architecture optimizes delivery speed, maintainability, and financial correctness over deep integration tests and production hardened security. Next phase strengthens security, reliability, and scalability.
- Known gaps:
  - Transfers idempotency implementation
  - Richer auth model
  - Deeper observability
  - Asynchronous queue ingestion
  - Worker-based settlement

Optimization Strategy

- **Reliability**: Enforce transfer uniqueness with a channel-provided idempotency reference. Reuse the same reference for safe retries, reversals, and downstream audit traceability.
- **Security**: Strengthen auth with API gateway controls (API keys, rate limits, geo/IP filtering), JWT claims for initiator context and authorization, and a dedicated channel-auth service.
- **Scalability**: Introduce Pub/Sub buffering for peak traffic and async transaction processing using either of Apache Kafka, RabbitMQ, GCP Pub/Sub, or AWS SQS.
- **Monitoring and Operations**: Improve observability with OpenTelemetry traces, Prometheus metrics, and Grafana dashboards.
- **Worker-based settlement:** Move fee settlement to asynchronous workers that consume queued transfer events, enabling retries, better fault isolation, and higher throughput under load.

Closing

- Built a reliable foundation for multi-currency internal and external transfers with auditable posting flows.
- Prioritized financial correctness and delivery speed under time constraints.
- Designed for maintainability through clean layering and repository boundaries.
- Next phase focuses on security, reliability, and scale: idempotency, async worker-based settlement, stronger auth, and deeper observability.